

已发交个朋友全栈AI工程师

一面

1. 谈谈你对浏览器事件循环机制的理解，特别是微任务在 AI 流式输出场景下的影响。

难度：★★

考点：Event Loop、微任务与宏任务、渲染时机

答案要点：

事件循环是浏览器协调脚本、渲染、网络等任务的机制，通过任务队列实现非阻塞 I/O。

- 宏任务包括 setTimeout、I/O、UI 渲染；微任务包括 Promise.then、MutationObserver。
- 每一轮宏任务执行完后，会清空所有微任务队列，随后决定是否进行 UI 渲染。
- 在 AI 聊天场景中，流式数据（Chunk）的接收通常在微任务中处理，频繁更新状态可能阻塞渲染。
- 需注意在微任务中进行大量 DOM 操作会导致页面掉帧，影响打字机效果的流畅度。

常见坑：

- 认为微任务和宏任务在同一队列。
- 忽略了 requestAnimationFrame 在渲染流程中的位置。

追问：

- 如果流式返回太快，如何优化 DOM 更新频率？

2. 在处理 AI 聊天流时，Fetch API 的 ReadableStream 是如何工作的？

难度：★★

考点：流式数据处理、Fetch API、Uint8Array 转换

答案要点：

ReadableStream 允许在资源完全下载前逐块读取数据，这对于大模型流式输出（SSE）至关重要。

- 通过 response.body.getReader() 获取读取器。
- 使用 reader.read() 循环读取数据块（Chunks）。
- 数据块通常是 Uint8Array，需要通过 TextDecoder 进行解码转为字符串。
- 必须处理流的关闭（done 状态）和异常中断。

常见坑：

- 忘记处理 TextDecoder 跨 Chunk 解码可能导致的乱码（中文字符被截断）。

- 没有正确关闭 reader 导致内存泄漏。

追问：

- 如何中断一个正在进行的 Fetch 请求？

3. 说一下 CSS 中实现一个类似 ChatGPT 自动滚到底部的方案。

难度：★

考点：CSS 布局、滚动锚定、Flexbox

答案要点：

实现自动滚动通常结合 CSS 布局特性与 JS 滚动 API，确保新消息出现时容器视口跟随。

- 使用 flex-direction: column-reverse 可以让内容从底部开始排列，但逻辑顺序会反转。
- 现代浏览器支持 overflow-anchor: auto 属性，可以实现滚动锚定。
- 传统方案是通过 JS 监听内容变化，调用 scrollToView({ behavior: 'smooth' })。
- 容器高度设置 flex: 1 并配合 overflow-y: auto 是基础。

常见坑：

- 用户手动向上滚动时，代码强制滚回底部会导致体验极差。
- 忽略了图片加载或 MathJax 渲染异步增加高度导致的滚动失效。

追问：

- 如何判断用户当前是否正处于“手动查看历史消息”的状态？

4. TypeScript 中的 interface 和 type 有什么区别？在定义 AI 插件协议时你会选哪个？

难度：★★

考点：TS 类型系统、扩展性、工程实践

答案要点：

interface 主要用于定义对象形状且支持声明合并，type 则是类型别名，支持联合类型和元组等。

- interface 可以被多次声明并自动合并，适合定义库的公共 API。
- type 支持更复杂的逻辑，如联合类型 (A | B)、交叉类型、映射类型。
- 在定义 AI 插件协议（如 Tool Call 结构）时，通常选 type，因为需要利用联合类型区分不同的工具参数。
- interface 在性能上略优于 type，因为 TS 编译器对 interface 有缓存优化。

常见坑：

- 混淆了 interface 的继承 (extends) 与 type 的交叉 (&)。
- 在需要导出可扩展配置时误用了 type。

追问：

- 什么是 TS 的类型守卫 (Type Guards)，在处理大模型返回的 JSON 时怎么用？

5. 手写一个简单的防抖函数（Debounce），并解释它在 AI 搜索建议场景下的作用。

难度：★

考点：闭包、高阶函数、性能优化

答案要点：

防抖是指在事件触发后延迟执行，若在延迟时间内再次触发则重新计时。

- 核心利用闭包保存 timer 变量。
- 每次触发先 clearTimeout。
- 在 AI 搜索建议中，防止用户每输入一个字符就调用一次昂贵的 LLM 接口。
- 减少服务器压力，提升前端响应性能。

常见坑：

- 忘记处理 this 指向和参数传递。
- 没考虑到组件卸载时需要清除定时器。

追问：

- 节流（Throttle）和防抖的区别是什么？

代码块

```
1  function debounce(fn, delay) {
2    let timer = null;
3    return function(...args) {
4      if (timer) clearTimeout(timer);
5      timer = setTimeout(() => {
6        fn.apply(this, args);
7      }, delay);
8    };
9  }
```

6. 介绍一下 SSE（Server-Sent Events）与 WebSocket 的区别，为什么 AI 聊天多采用 SSE？

难度：★★

考点：网络协议、长连接、AI 场景适配

答案要点：

SSE 是基于 HTTP 的单向推送协议，而 WebSocket 是全双工的独立协议。

- SSE 更加轻量，直接复用 HTTP 基础设施，支持自动重连。
- WebSocket 支持双向通信，但协议握手复杂，对代理服务器不友好。
- AI 场景主要是模型单向输出文本，SSE 的单向流特性完全满足需求。

- SSE 默认支持断线重连和事件 ID，方便处理长文本生成中断。

常见坑：

- 认为 SSE 只能传输文本（其实可以通过 Base64 传二进制）。
- 忽略了 HTTP/1.1 下浏览器对同一域名的 SSE 连接数限制（通常为 6 个）。

追问：

- 在 HTTP/2 下，SSE 的连接限制还存在吗？

二面（深入原理与项目）

1. React 的 Fiber 架构是如何解决复杂 UI 渲染卡顿问题的？在长对话列表里怎么应用？

难度：★★★

考点：React 原理、并发模式、调度算法

答案要点：

Fiber 将渲染工作拆分为微小的任务单元，通过时间切片（Time Slicing）避免阻塞主线程。

- 引入了双缓存技术和可中断的更新机制。
- 协调阶段（Reconciliation）变为异步可中断，提交阶段（Commit）仍是同步的。
- 在长对话列表中，利用 React 18 的 `useTransition` 或 `useDeferredValue` 降低非紧急更新的优先级。
- 配合虚拟列表（Windowing）只渲染可视区域，减少 DOM 节点数。

常见坑：

- 误以为 Fiber 提高了单次渲染的速度（实际上是提高了响应性）。
- 在循环中错误使用 key 导致 Fiber 树频繁销毁重建。

追问：

- 简述一下 diff 算法在 Fiber 架构中的变化。

2. 在前端如何处理大模型返回的 Markdown 实时渲染？特别是包含代码高亮和数学公式时。

难度：★★★

考点：Markdown 解析、正则处理、性能优化

答案要点：

实时渲染需要平衡解析速度与显示效果，通常采用增量解析或高性能解析库。

- 使用 `markdown-it` 或 `marked` 作为基础解析器。
- 代码高亮集成 `Prism.js` 或 `lowlight`，数学公式使用 `KaTeX` 或 `MathJax`。

- 针对流式输出，需要处理“非完整 Markdown 语法”的中间状态（如未闭合的代码块）。
- 性能优化：使用 memo 缓存已解析的片段，避免全文重新渲染。

常见坑：

- 每次 Chunk 更新都全量解析整个对话字符串，导致输入后期极度卡顿。
- XSS 攻击风险，未对大模型输出的 HTML 标签进行过滤。

追问：

- 如果模型返回了一个超长的表格，解析过程阻塞了 UI 怎么办？

3. 谈谈 Prompt Engineering 的常用技巧，前端如何在代码层面对 Prompt 进行封装？

难度：★★

考点：提示词工程、模板化、上下文管理

答案要点：

提示词工程是通过优化输入来提升模型输出质量的技术，核心在于清晰、具体、有上下文。

- 常用技巧：Few-shot（少样本提示）、CoT（思维链）、Role Play（角色扮演）。
- 前端封装：建立 Prompt Template 模板系统，通过变量注入用户输入。
- 结构化输出：在 Prompt 中指定 JSON Schema，确保前端能稳定解析返回数据。
- 敏感词过滤：在发送给模型前进行前端预审。

常见坑：

- 将敏感业务逻辑直接写在前端 Prompt 中（容易被 Prompt Injection 破解）。
- 忽略了 Token 长度限制，没有做上下文裁剪。

追问：

- 什么是 Prompt Injection（提示词注入），如何防范？

4. 什么是 RAG（检索增强生成）？请描述一个前端参与的 RAG 完整流程。

难度：★★★

考点：RAG 架构、向量检索、前端交互

答案要点：

RAG 通过检索外部知识库来弥补大模型训练数据的时效性和幻觉问题。

- 流程：用户提问 -> 文本向量化（Embedding） -> 向量数据库检索相关片段 -> 拼接 Prompt -> LLM 生成回答。
- 前端职责：处理文档上传、切片预览、展示检索来源（Citations/References）。
- 交互设计：在回答中高亮显示引用的原始文档片段。
- 性能：前端可以做初步的语义搜索缓存。

常见坑：

- 检索回来的片段过多，导致 Prompt 超过 LLM 的 Context Window。
- 忽略了检索片段与生成内容的相关性校验。

追问：

- 向量数据库你了解哪些？前端如何调用 Embedding 接口？

5. 如何实现一个 AI Agent 的工具调用（Function Calling）流程？

难度：★★★

考点：Agent 逻辑、JSON Schema、状态机

答案要点：

Function Calling 允许模型根据需求调用外部 API，实现从“思考”到“行动”的跨越。

- 步骤：定义工具函数描述（JSON Schema） -> 发送给 LLM -> LLM 返回 call_info -> 前端/后端执行本地函数 -> 将结果回传给 LLM。
- 前端需要维护一个“执行中”的状态，展示工具调用的进度（如“正在搜索天气...”）。
- 异常处理：如果工具执行失败，需要将错误信息反馈给模型让其重试。
- 安全性：严格校验模型返回的参数类型，防止执行恶意代码。

常见坑：

- 递归调用陷阱：模型不断调用工具进入死循环。
- 忽略了多工具并行调用的场景。

追问：

- 怎么在 UI 上优雅地展示 Agent 的思考过程（Thought Process）？

6. Node.js 作为 AI 中间层时，如何处理高并发的流式请求？

难度：★★★

考点：Node.js Stream、背压（Backpressure）、内存管理

答案要点：

Node.js 适合处理 I/O 密集型任务，通过 Stream 模块可以高效转发 AI 厂家的流式响应。

- 使用 pipeline 或 stream.pipe() 建立从 AI API 到客户端的管道。
- 处理背压：当客户端消费速度慢于 AI 产出速度时，暂停读取上游流。
- 使用中间层进行 Token 计数、日志记录和权限校验。
- 部署：利用 PM2 或 Docker 容器化，结合 Nginx 进行负载均衡。

常见坑：

- 在中间层将流转为字符串处理，导致大并发下内存直接爆满。
- 忘记设置合理的超时时间，导致僵尸连接占用资源。

追问：

- 如何在 Node.js 中实现对不同用户的流式请求限流？

7. 谈谈你对大模型“幻觉”问题的理解，前端有哪些手段可以辅助缓解？

难度：★★

考点：LLM 局限性、用户体验设计

答案要点：

幻觉是指模型生成事实错误或逻辑不通的内容，本质是概率预测的偏差。

- UI 手段：增加“内容由 AI 生成，请核实”的提示。
- 交互手段：提供一键搜索验证功能，或展示 RAG 检索的来源依据。
- 约束手段：通过 System Prompt 强制模型在不确定时回答“不知道”。
- 反馈机制：设计点赞/点踩功能，收集数据用于后续微调或 Prompt 优化。

常见坑：

- 过度信任模型输出，直接将其作为业务逻辑的唯一判断依据。

追问：

- 什么是 Temperature 参数，它如何影响幻觉的产生？

8. 在 Next.js 项目中，如何优化 AI 应用的 FCP 和 TTI 指标？

难度：★★★

考点：Next.js 优化、SSR/ISR、流式渲染

答案要点：

优化 AI 应用需结合 Next.js 的服务端特性与流式输出特点。

- 使用 Streaming SSR (React Suspense)：先渲染页面骨架，再流式传输 AI 内容。
- 静态内容使用 ISR (增量静态再生)，动态对话部分走客户端 fetch。
- 优化字体和图片加载，减少阻塞渲染的 JS 体积。
- 利用 Edge Runtime 部署 AI 接口，减少地理位置带来的网络延迟。

常见坑：

- 在服务端等待 AI 全量返回后再渲染，导致首屏白屏时间极长。
- 忽略了 Hydration 错误（服务端与客户端渲染内容不一致）。

追问：

- Edge Runtime 和 Node.js Runtime 有什么区别？

三面（架构与综合能力）

1. 如果让你从零设计一个企业级 AI Agent 平台的前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统设计、架构思维、可扩展性

答案要点：

一个成熟的 Agent 平台需要兼顾易用性、调试性和可观测性。

- 核心模块：可视化工作流编辑器（基于 React Flow）、Prompt 管理中心、插件/工具集市。
- 调试模块：支持单步执行、中间状态查看（Trace）、Token 消耗统计。
- 存储模块：多端同步的对话历史、用户偏好设置。
- 接入层：多模型适配层（统一不同厂商的 API 格式）。
- 监控层：错误上报、模型响应质量评估（Eval）。

常见坑：

- 架构过于耦合，导致更换底层模型或增加新工具时需要大规模重构。
- 忽视了私有化部署的需求，导致静态资源和 API 路径硬编码。

2. 在 AI 产品中，如何平衡“极致的交互体验”与“昂贵的 API 成本”？

难度：★★★

考点：成本控制、产品思维、技术选型

答案要点：

成本优化是 AI 工程师的核心能力，需从模型选择、缓存和请求策略入手。

- 模型分级：简单任务用轻量模型（如 GPT-4o-mini），复杂任务用旗舰模型。
- 语义缓存：对相似问题使用 Redis 缓存结果，避免重复调用。
- 长度控制：前端强制限制上下文轮数，或使用摘要算法压缩历史记录。
- 交互优化：通过乐观更新、预加载等手段掩盖模型响应的延迟。

常见坑：

- 盲目追求模型效果而忽略 Token 消耗，导致项目亏损。
- 缓存策略太激进，导致用户得到过时的信息。

3. 聊聊你在过去一年中遇到的最难的 AI 技术挑战，你是如何解决的？

难度：★★★

考点：问题解决能力、技术深度、复盘习惯

答案要点：

此题考察真实经验，需按 STAR 法则描述。

- 场景：例如处理超长文本的 Embedding 丢失、流式输出中的样式错乱、或者 Agent 逻辑死循环。
- 挑战：具体的技术瓶颈是什么？为什么常规方案不行？
- 行动：尝试了哪些方案？如何进行压力测试或对比实验？

- 结果：最终量化的指标提升（如延迟降低 30%，准确率提升 20%）。

常见坑：

- 描述过于笼统，缺乏具体的技术细节。
- 解决方案全是“调包”，没有体现自己的思考和优化。

4. 你认为未来 1-2 年，前端工程师在 AI 浪潮下的核心竞争力是什么？

难度：★★

考点：行业视野、职业规划

答案要点：

前端工程师应从“UI 实现者”转型为“AI 能力的整合者”。

- 熟练掌握 AI 基础设施：如 LangChain、LlamaIndex、向量数据库的使用。
- 交互创新能力：探索除聊天框以外的 AI 交互形态（如 Copilot 模式、画布模式）。
- 全栈化能力：能够编写高效的中间层代码，处理复杂的 AI 业务逻辑。
- 评估与调优能力：能够科学地评估 Prompt 效果并进行工程化改进。

常见坑：

- 表现出对 AI 的焦虑但没有实际行动。
- 认为 AI 只是换个接口调用，忽视了底层原理的学习。

5. 假如公司要自研一个低代码 AI 工作流工具，你会如何设计其数据结构来支持复杂的嵌套逻辑？

难度：★★★

考点：数据建模、图论基础、低代码原理

答案要点：

低代码工具的核心是 DSL（领域特定语言）的设计。

- 采用 DAG（有向无环图）结构存储节点和边。
- 节点定义：包含类型（模型、工具、判断、输入/输出）、配置参数、输入输出 Schema。
- 状态传递：设计全局变量与局部变量作用域，支持上游节点输出作为下游输入。
- 序列化：DSL 需支持 JSON 格式，方便版本管理和后端解析执行。

常见坑：

- 没考虑循环引用问题。
- 没考虑异步节点执行时的状态同步。

6. 如果团队内部对技术选型（如使用 React 还是 Next.js）产生严重分歧，你作为核心开发会如何决策？

难度：★★

考点：沟通协作、技术决策、领导力

答案要点：

决策应基于业务目标而非个人喜好。

- 建立评价维度：开发效率、性能指标（FCP/SEO）、维护成本、团队技术储备。
- 灰度实验：针对核心痛点（如 AI 渲染延迟）做 Demo 对比测试。
- 达成共识：组织技术评审会，陈述利弊，最终少数服从多数并全力执行。
- 风险预案：为选定的方案准备好应对潜在问题的兜底方案。

常见坑：

- 过于强势或过于软弱，导致团队氛围受损。
- 决策缺乏数据支持，全凭感觉。